人工智能第一次作业
拼音输入法实验报告

钟锦程　2026210748　电子工程系

2026 年秋

目录

1　实验环境 **2**

2　语料库与数据预处理 **3**

3　基于字的二元模型拼音输入法 **3**

4　Viterbi 算法 **5**

5　实验效果 **6**

6　改进方法：句首统计 **7**

7　思考题 **7**

8　总结与感受 **8**

1 实验环境

1.1 软硬件环境

实验在 macOS 26.6.2 上完成，处理器为 Apple M5 Pro，内存为 24 GiB，Python 版本为 3.13.13。编辑器使用 Visual Studio Code，LaTeX 报告使用 XeLaTeX 编译。程序使用 Python 标准库，主要包括 `json`、`collections`、`math`、`pickle`、`gzip` 和 `pathlib`。

1.2 GBK 读取与 UTF-8 输出

新浪新闻语料、拼音汉字表和一二级汉字表采用 GBK 编码，所以打开文件时要把编码写清楚。这一部分是我在实验开始时遇到的第一个问题：vscode 默认使用 UTF-8 打开 GBK 文件，显示的中文是乱码。这可以通过在项目根配置文件`.vscode/settings.json` 中加入

```
{
  "files.autoGuessEncoding": true,
  "files.candidateGuessEncodings": ["gb2312"]
}
```

来解决。

我在程序中统一使用 `gb18030` 读取这些文件：

```
with path.open(encoding="gb18030") as source:
    for line in source:
        news = json.loads(line)
```

GB18030 兼容 GBK，可以按照 GBK 的字节规则解码本实验文件，同时能覆盖更多汉字。读取过程可以分为两步：系统先从文件中读出原始字节，然后 Python 根据 `gb18030` 将字节转换为 `str`。从这一步开始，程序中的“清”、“华”等字就是 Unicode 字符，词频统计可以直接以它们为字典的键。

训练完成后，程序通过标准输出打印预测句子。macOS 终端环境的默认文本编码为 UTF-8，因此下面的输出重定向会得到 UTF-8 文件：

```
python main.py < data/input.txt > data/output.txt
```

评测程序在保存结果时也显式指定 UTF-8：

```
output_path.write_text(text, encoding="utf-8")
```

整个转换过程可以写成：

$$\text{GBK/GB18030 字节} \longrightarrow \text{Python Unicode 字符串} \longrightarrow \text{UTF-8 字节}。$$

2 语料库与数据预处理

2.1 语料库

使用附件中的新浪新闻 2016 年 4 月至 11 月语料，共 8 个 JSON Lines 文件。每一行都是一个独立的 JSON 对象，其中 `title` 字段保存新闻标题，`html` 字段保存新闻正文。我从这两个字段中统计汉字的出现次数和相邻关系。

拼音汉字表给出了每个拼音对应的候选字。例如，输入 `qing` 时会得到“清”、“情”、“请”、“庆”等候选字。一二级汉字表给出本实验使用的 6763 个汉字，训练时的统计范围以这张字表为准。

2.2 预处理和词频统计

预处理的步骤如下：

1. 读取一二级汉字表，将 6763 个汉字放入集合 `allowed` 中。集合可以快速判断当前字是否属于统计范围。
2. 按文件名顺序打开 8 个新闻文件，按行读取和解析 JSON。
3. 依次扫描每条新闻的标题和正文。当字符属于 `allowed` 时，将该字的一元计数加 1。
4. 用变量 `previous` 记录前一个合法汉字。当当前字和前一字都存在时，将它们组成二元组，对该二元组的计数加 1。
5. 扫描到标点、数字、英文和字表外字符时，将 `previous` 设回空值。下一个合法汉字会成为新文本片段的开头。这样的统计更符合句子内部的相邻关系。
6. 对每个文本片段的首字进行计数，这个计数会用在后面的句首统计改进中。

一元词频用 `Counter` 保存，形式可以理解为“汉字 $\rightarrow$ 出现次数”。二元词频用两层映射保存，形式可以理解为“前一个字 $\rightarrow$ 后一个字 $\rightarrow$ 相邻出现次数”。统计完成后，程序使用 `pickle` 序列化这些字典，再使用 `gzip` 压缩为模型文件。

3 基于字的二元模型拼音输入法

3.1 从输入法问题开始建模

一个拼音会对应很多汉字。如果每个拼音都单独选词频最高的字，就会忽略字与字之间的联系。如何建模这种联系，个人认为是输入法问题中的基本问题。例如“清华大学”中的“清华”和“大学”在文本中经常相邻出现，这种上下文信息可以帮助我们从同音字中做选择，用简单的条件概率模型就可以体现这种联系。

设输入的拼音序列为

$$Y = (y_1, y_2, \ldots, y_n), \tag{1}$$

对应的汉字序列为

$$X = (x_1, x_2, \ldots, x_n). \tag{2}$$

拼音表为每个 y_i 给出候选字集合 $S(y_i)$，所以每个位置都要满足 $x_i \in S(y_i)$。输入法的目标是在所有符合拼音的汉字序列中，找出在中文里最可能出现的一句：

$$X^* = \arg\max_{x_i \in S(y_i)} P(X \mid Y). \tag{3}$$

在候选字集合已经由拼音确定后，我们可以对各条合法汉字序列的语言概率 $P(X)$ 进行比较：

$$X^* = \arg\max_{x_i \in S(y_i)} P(x_1, x_2, \ldots, x_n). \tag{4}$$

3.2 从联合概率到二元模型

根据概率的链式法则，一句话的联合概率可以展开为

$$\begin{aligned} P(x_1, x_2, \ldots, x_n) = P(x_1)P(x_2 \mid x_1)P(x_3 \mid x_1, x_2) \\ \cdots P(x_n \mid x_1, x_2, \ldots, x_{n-1}). \end{aligned} \tag{5}$$

直接统计很长的历史序列需要大量语料，同时依赖高维度的条件概率。这里我们使用二元模型，做了一个简化：当预测第 i 个字时，只使用第 $i-1$ 个字作为上下文。数学上可以写成

$$P(x_i \mid x_1, \ldots, x_{i-1}) \approx P(x_i \mid x_{i-1}). \tag{6}$$

于是整句概率可以近似为

$$P(X) \approx P(x_1) \prod_{i=2}^{n} P(x_i \mid x_{i-1}). \tag{7}$$

这个模型使用了相邻字的关系，所需的词频表也可以从新闻语料中直接统计，比较方便实现。

3.3 由词频估计概率

设语料中共有 N 个合法汉字，字库中共有 V 个不同汉字。$N(c)$ 表示字 c 的出现次数，$N(a, c)$ 表示 a 和 c 相邻出现的次数。一元概率的直观想法是用出现次数除以总字数。程序中加入加一平滑，得到

$$P_1(c) = \frac{N(c) + 1}{N + V}. \tag{8}$$

分子给每个字增加 1 次，分母相应增加 V。这使得语料中出现次数为 0 的候选字也有一个很小的正概率。

在前一个字为 a 的条件下，后一个字为 c 的频率估计为

$$P_2(c \mid a) = \frac{N(a, c)}{N(a)}. \tag{9}$$

某些合理的字组在训练语料中可能没有出现。我用线性插值把二元概率和一元概率结合：

$$\widehat{P}(c \mid a) = \lambda P_2(c \mid a) + (1-\lambda) P_1(c), \tag{10}$$

本实验取 $\lambda = 0.99$。二元统计在打分中占主要位置，一元统计提供平滑概率。最终要最大化的句子概率为

$$\widehat{P}(X) = P_1(x_1) \prod_{i=2}^{n} \widehat{P}(x_i \mid x_{i-1}). \tag{11}$$

长句中会出现很多小概率相乘。程序对概率取对数，利用对数的单调性和乘法转加法性质：

$$\begin{aligned} \arg\max_X \widehat{P}(X) &= \arg\max_X \log \widehat{P}(X), \\ \log \widehat{P}(X) &= \log P_1(x_1) + \sum_{i=2}^{n} \log \widehat{P}(x_i \mid x_{i-1}). \end{aligned} \tag{12}$$

这样一条路径的分数可以通过逐步相加得到，也很适合后面的动态规划。

4 Viterbi 算法

4.1 为什么可以使用 Viterbi

假设一个拼音平均有 K 个候选字，长度为 n 的输入大约会产生 K^n 个候选句子。逐句枚举会随着输入长度快速增长。

二元模型带来了一个很有用的性质：已经到达当前字 c 时，下一步的转移概率由 c 和下一字决定。两条路径到达同一个 c 后，它们面对的后续选择相同。此时保留累计分数更高的那条路径，它在加上任意相同后续分数后依然更高。这就是动态规划的最优子结构，也是 Viterbi 算法能用在本问题中的原因。

可以把输入看成一张分层图。第 i 层对应拼音 y_i，该层的每个结点是 $S(y_i)$ 中的一个候选字。相邻两层候选字 a 和 c 之间的边权为 $\log \widehat{P}(c \mid a)$。拼音输入法就变成了在这张分层图中寻找总分最高的路径。

4.2 递推公式

用 $\delta_i(c)$ 表示“前 i 个拼音已处理，并且第 i 个字为 c”时能得到的最高对数分数。第一层的初始化为

$$\delta_1(c) = \log P_1(c), \qquad c \in S(y_1). \tag{13}$$

对于第 i 层的候选字 c，程序尝试第 $i-1$ 层的每个候选字 a，并选取累计分数最高的一个：

$$\delta_i(c) = \max_{a \in S(y_{i-1})} \left[\delta_{i-1}(a) + \log \widehat{P}(c \mid a) \right]. \tag{14}$$

在计算最大值的同时，用 $\psi_i(c)$ 记录取到该最大值的前驱字：

$$\psi_i(c) = \arg\max_{a \in S(y_{i-1})} \left[\delta_{i-1}(a) + \log \widehat{P}(c \mid a) \right]. \tag{15}$$

所有拼音处理完后，在第 n 层找到分数最高的终点：

$$x_n^* = \arg\max_{c \in S(y_n)} \delta_n(c). \tag{16}$$

然后按照前驱记录从后向前回溯：

$$x_{i-1}^* = \psi_i(x_i^*), \qquad i = n, n-1, \ldots, 2. \tag{17}$$

以 `qing hua da xue` 为例，程序会先为 `qing` 的每个候选字计算初始分数。到 `hua` 这一层时，“华”会尝试与“清”、“情”、“庆”等前驱连接，并记录得分最高的前驱。后面的 `da` 和 `xue` 使用同样的处理。达到最后一层后，程序从终点沿着前驱记录返回，得到“清华大学”。

5 实验效果

测试集共有 501 句、5235 个汉字。本节汇报下面三项实验指标。

1. **生成词频表的时间和大小**：完整扫描 8 个新浪新闻文件并生成模型用时 98.91 秒，经 `gzip` 压缩后的词频模型大小为 8.86 MiB。
2. **给定测试样例上的准确率**：最终程序的字准确率为 83.90% ，句准确率为 40.12% 。
3. **生成全部测试样例的总时间**：已加载词频模型的情况下，生成 501 句预测结果共用时约 0.9 秒。

5.1 效果较好的例子

输入 `qing hua da xue` 时，程序输出“清华大学”；输入 `ren gong zhi neng` 时，程序输出“人工智能”。这些词组在新闻语料中经常出现，“清华”、“华大”、“大学”以及“人工”、“工智”、“智能”的二元词频可以在各层候选中给正确路径较高的分数。

后面经过简单优化的句首统计方法对部分句子也有帮助。例如 `de guo zong li` 通过普通一元词频选首字时容易得到“的国总理”，句首统计给“德”更高的初始分数，结果变为“德国总理”。`dai kou zhao` 也从“大口罩”变为“戴口罩”。

5.2 效果较差的例子

输入 `chuang qian ming yue guang` 时，标准答案为“床前明月光”，程序输出“创千名越广”。新浪新闻中古诗文本的比例较少，“床前”、“明月”这样的搭配优势没有充分体现在统计中。

输入 `sheng hua huan cai` 时，标准答案为“生化环材”，程序输出“省华垸蔡”。“生化环材”是一个缩略语，每两个相邻字在普通新闻中的共现次数较少。二元模型每次使用前一个字，固定短语和缩略语带来的长距离信息在模型中体现得较弱。

6 改进方法：句首统计

我在检查错误样例时发现，一些错误出现在第一个字。例如“的”在全部新闻中的频率很高，它在普通句子的开头位置频率较低。于是我在预处理时额外统计了每个文本片段的首字次数 $N_s(c)$。

设所有文本片段的数量为 N_s，句首概率使用加一平滑：

$$P_s(c) = \frac{N_s(c) + 1}{N_s + V}. \tag{18}$$

Viterbi 的第一层使用 $\log P_s(c)$ 作为初始分数，后面各层继续使用前面的二元转移概率。这个改动需要多保存一张句首计数表，Viterbi 的递推过程保持原样。从具体例子可以看到，它能够利用“哪些字常出现在句首”这条额外信息。

7 思考题

7.1 GBK、UTF-8 与其他编码

GBK 是面向简体中文的字符编码，它兼容 GB2312。在 GBK 中，ASCII 字符通常占 1 个字节，常用汉字通常占 2 个字节。很多较早的中文 Windows 软件和中文数据集使用 GBK，本实验的新浪新闻语料就是一个例子。

UTF-8 是 Unicode 的可变长编码。ASCII 字符在 UTF-8 中占 1 个字节，常用中文汉字通常占 3 个字节。Unicode 为不同语言的字符分配统一的码点，UTF-8 再把这些码点表示成字节。网页、Linux/macOS 终端、Python 源代码和很多现代文本文件都广泛使用 UTF-8。

同一个汉字在 GBK 和 UTF-8 中会对应不同的字节序列。用 UTF-8 规则解释 GBK 字节时，编辑器得到的字符会发生错误，就会看到乱码或替换符号。因此了解文件原始编码很重要。

常见的其他编码还有：

- **UTF-16**：常用字符主要使用 2 个字节的码元，部分字符使用两个码元。Windows 的一些系统接口和 Java 字符串内部表示与 UTF-16 关系密切。
- **ASCII**：表示英文字母、数字、常用符号和控制字符，适合英文配置文件和简单文本协议。
- **Big5**：用于繁体中文文本，在早期的中国台湾和中国香港软件、网页中较常见。

7.2 Viterbi 算法的时间和空间复杂度

设输入句子有 n 个拼音，字库中有 V 个汉字，拼音的种类数为常数 C。题目假设汉字在读音上均匀分布，因此每个拼音平均对应的候选字数为

$$K = \frac{V}{C}. \tag{19}$$

先分析时间复杂度。第一层有 K 个候选字，计算它们的初始分数需要 $O(K)$ 时间。从第二层开始，每一层有 K 个当前候选字。对每个当前字，程序要检查前一层的 K 个候选字，因此一层需要检查 $K \times K = K^2$ 条转移。这样的转移层共有 $n - 1$ 层，总时间为

$$
\begin{aligned}
T(n,V,C) &= O(K) + O((n-1)K^2) \\
&= O\left(n\left(\frac{V}{C}\right)^2\right).
\end{aligned} \tag{20}
$$

题目已经设定 C 是常数，在大 O 记号中可以将 $1/C^2$ 省略，因此时间复杂度还可以写为

$$
O(nV^2). \tag{21}
$$

再分析空间复杂度。计算当前层分数时，程序需要保存前一层和当前层的分数，两层都是 K 个数，这部分为 $O(K)$。为了在结束时恢复整句，程序要给每一层的每个候选字保存一个前驱字，共需要 nK 个前驱记录。因此总空间为

$$
\begin{aligned}
S(n,V,C) &= O(K) + O(nK) \\
&= O\left(n\frac{V}{C}\right) \\
&= O(nV),
\end{aligned} \tag{22}
$$

其中最后一步同样用到了 C 为常数的条件。

7.3 两种输入输出代码

代码 A 在每次读入一个数后立即输出该数，因此输入 1 后会立即看到 1，接着输入 2 后会立即看到 2。它按照读入顺序输出数据，最终标准输出为两行“1”和“2”，所以代码 A 正确。

代码 B 在读取阶段把 1 和 2 依次加入列表。输入结束后，程序遍历列表并按原来的顺序输出，最终标准输出同样为两行“1”和“2”，所以代码 B 也正确。

两段代码的主要区别在于输出时刻和内存使用。代码 A 边读边输出，代码 B 在读取结束后统一输出，并且需要列表保存已读入的数据。本题是普通的批处理 OJ，评测器在程序结束后比较完整输出，因此两种写法都能得到标准答案。

8 总结与感受

本科期间学数字通信的时候就用到了 Viterbi 算法。那时的应用是信道编码，输入是接收的信号，输出是最可能的码字序列。这个实验中，Viterbi 算法被用在拼音输入法上，输入是拼音序列，输出是最可能的汉字序列。转码之后，了解到这是动态规划的一个典型应用。核心是最优子结构这个概念。

一些实现上的小细节也很明显地影响实验效果。最开始提交 OJ 的时候，单字概率限制在同音字内部计算。其实乍一看，单字概率用同拼音的字集中的概率似乎更合理：我都已经知道这个单字叫什么了，当然从这个拼音的同音字中选一个概率最高的字更合理。但是在同拼音集上的概率一般很大，相比于二元概率来说一般要大一个数量级。而我们本来设计的一元-二元平滑公式就是为了让二元概率占主导，把 λ 设的很接近 1. 现在单字概率比二元概率大一个数量级，相当于隐式的将 λ 调小了，一元概率的主导地位上升了。这样就会导致二元概率的作用被削弱，最终导致准确率下降。

现在把一元概率放回整个汉语语料中计算，OJ 里的正确率从 27.4 提高到 37.4，说明我们上述分析是有一定道理的。